Лабораторная работа 2: параллельное блочное MMM

План эксперимента и ответы на вопросы

1. Три ключевых числа плюс параллелизм

Все замеры при N = 2000, r = 50, если не указано иначе. Каждый следующий вариант добавляет ровно одну оптимизацию к предыдущему, чтобы изолировать её вклад.

1.1. Число 1: вклад порядка циклов

Берём наивный i-j-k и меняем порядок на i-k-j, ничего больше не трогая. Оба собираем с -O2 без -march=native, чтобы исключить SIMD.

1. baseline – i-j-k, O2, scalar.
2. ikj – i-k-j, O2, scalar.

Формула: E1 = T(baseline) / T(ikj).

Что измеряем: чистый эффект locality. Меняется только паттерн доступа к памяти, инструкций столько же.

1.2. Число 2: вклад блокировки

Берём i-k-j из шага 1 и добавляем блочную структуру (внешние циклы ii, jj, kk). Собираем с -O2 без -march=native.

1. ikj – i-k-j, O2, scalar, без блокировки.
2. ikj_block – i-k-j + blocking, O2, scalar.

Формула: E2 = T(ikj) / T(ikj_block).

Что измеряем: чистый эффект блокировки. Порядок циклов тот же, SIMD нет, меняется только размер рабочего набора.

1.3. Число 3: вклад SIMD

Берём лучший вариант из шага 2 и пересобираем с -O3 -march=native. Порядок циклов и блокировка не меняются, меняется только набор инструкций (AVX2/FMA).

1. ikj_block – O2, scalar.
2. ikj_block_avx – O3, AVX2.

Формула: E3 = T(ikj_block) / T(ikj_block_avx).

Что измеряем: чистый эффект векторизации. GCC сам решит, что можно векторизовать. У внутреннего цикла по j (row-wise по ans и b) шансы высокие.

1.4. Проверка мультипликативности

Если три эффекта независимы, то:

T(baseline) / T(ikj_block_avx) примерно равно E1 **E2** E3.

Если реальное произведение больше – эффекты перекрываются, что-то мешает друг другу (например, bandwidth или throttling). Если меньше – есть синергия (маловероятно для этих оптимизаций).

Дополнительный контроль: собрать baseline с -O3 -march=native. Если AVX2 на нём ничего не даёт – значит, наивный i-j-k не векторизуется (ожидаемо, column-wise b), и эффект SIMD действительно проявляется только при правильном порядке циклов.

1.5. Отдельно: эффект параллелизма

Параллелизм – другая ось, он борется не с cache misses, а с простаивающими ядрами. Поэтому измеряем его отдельно, поверх лучшего однопоточного варианта.

1. par1 – ikj_block_avx с pragma omp parallel for на внешнем цикле ii.
2. par2 – то же с pragma omp parallel for collapse(2) на ii и jj.

Для p = 1, 2, 4, 6, 8. Для каждого p берём min из 5 после warm-up.

Формула: Ep(p) = T(ikj_block_avx, 1) / T(par, p).

Что измеряем: масштабирование. Отклонение Ep(p) от p показывает вклад throttling, bandwidth saturation и scheduling expenses.

1.6. Проверка независимости блокировки и параллелизма

Сравниваем два отношения:

1. R1 = T(baseline, 1) / T(ikj_block_avx, 1) – эффект однопоточных оптимизаций.
2. R2 = T(baseline, p) / T(ikj_block_avx, p) – тот же эффект, но на p потоках.

Если R2 примерно равно R1 – блокировка и параллелизм независимы. Если R2 меньше – эффекты перекрываются (например, оба упираются в DRAM bandwidth). Если больше – маловероятно, но может быть из-за лучшей утилизации памяти при параллелизме.

1.7. Дополнительные измерения

1. perf stat -e cache-misses,cache-references,LLC-load-misses на всех вариантах. Показывает, ушла ли память из узкого места после блокировки.
2. perf stat -e instructions,cycles – IPC. Растёт с порядком циклов и SIMD, падает при memory bound.
3. watch -n 0.5 ‘grep MHz /proc/cpuinfo | head −8’ параллельно с бенчмарком – ловим throttling.
4. objdump -d на каждом бинарнике – убедиться, что циклы не переставлены и что AVX2 действительно применён там, где мы думаем.

1.8. Ожидаемые значения для N = 2000

Точные числа зависят от машины, но порядок такой:

1. E1 (порядок циклов) – 1.5-3x. Меньше, чем на маленьких N, потому что при N=2000 матрицы в DRAM и bandwidth ограничивает.
2. E2 (блокировка) – 2-4x. Основной эффект: локальность рабочего набора.
3. E3 (SIMD) – 2-4x. AVX2 даёт 8 int за инструкцию.
4. Произведение E1 **E2** E3 – 8-30x полного speedup от baseline к лучшему однопоточному.
5. Ep(8) – 3-4x. Меньше 8 из-за throttling, bandwidth и Amdahl.

1.9. Ответы на вопросы из лабы

1.10. Число потоков

p задаётся через OMP_NUM_THREADS или omp_set_num_threads. Набор p = 1, 2, 4, 6, 8 покрывает случаи меньше, равно и больше числа физических ядер (у Ryzen 7 7735HS – 8 физических, 16 логических). Это позволяет увидеть, где линейное масштабирование заканчивается и начинается плато.

1.11. Выбор циклов для образования потоков вычислений

Два разумных варианта и один плохой.

Вариант 1: pragma на внешнем цикле ii. Каждый поток получает свой набор блоков по строкам, пишет в свою часть ans, гонки нет. Итераций при N=2000, r=50 около 40 – для 8 потоков хватает.

Вариант 2: pragma collapse(2) на ii и jj. Объединяет два цикла в один плоский, итераций становится 1600. Больше параллелизма, но и больше накладных на распределение.

Плохой вариант: pragma на внутреннем цикле i или kk. Параллелить kk нельзя – разные потоки пишут в одну ans[i*N+j], это гонка. Параллелить i можно, но pragma срабатывает на каждой итерации блочных циклов, и OpenMP создаёт и уничтожает команду потоков десятки тысяч раз. Накладные съедают весь выигрыш.

1.12. Scheduling expenses

Накладные расходы OpenMP на создание потоков, распределение итераций и синхронизацию. Проявляются так:

1. при p=2 ускорение заметно меньше двух – фиксированная часть overhead сравнима с полезной работой.
2. ускорение выходит на плато раньше, чем кончаются ядра – растёт доля последовательного кода.
3. при p больше числа физических ядер ускорение не растёт и может падать.

Снизить overhead: schedule(static) для равномерных итераций, не создавать параллельную область на каждой итерации внешнего цикла, применять collapse только когда итераций мало.

1.13. Memory bound

Признаки:

1. высокая доля cache-misses к cache-references в perf.
2. IPC меньше единицы.
3. добавление потоков почти не ускоряет – все ждут ту же память.
4. добавление блокировки, наоборот, ускоряет сильно.

Проверка: у baseline ожидаем десятки процентов cache-misses. У блочного – единицы процентов. Если после блокировки misses упали, но время не упало при росте p, значит упёрлись в пропускную способность DRAM, а не в кэш.

1.14. Throttling

Мобильный Ryzen 7 7735HS снижает частоту при длительной нагрузке на все ядра. Один поток – 4.5-4.8 ГГц, восемь – 2.5-3.5 ГГц. Поэтому даже идеально параллельный

код даст не p-кратное ускорение, а p, умноженное на f(p)/f(1). При f(8)/f(1) примерно 0.5 получим speedup около 4, а не 8.

1.15. Что должно получиться в итоге

1. Три ключевых числа E1, E2, E3 – вклад порядка циклов, блокировки, SIMD соответственно. Каждое изолировано: меняется ровно одна вещь.
2. Произведение E1 **E2** E3 – полный однопоточный speedup относительно baseline.
3. Ep(p) – параллельное масштабирование, отдельно от однопоточных оптимизаций.
4. Проверка независимости блокировки и параллелизма – сравнение R1 и R2 при одном и том же p.
5. Степени двойки (N=2048) медленнее N=2000 из-за conflict misses, но блокировка частично лечит их.